Robô de cargas automáticas do Vecton — resumo para a TI

Data: 15/10/2026 · **Responsável pelo projeto:** Ricardo Guimarães (Controladoria)
Código: repositório GitHub `AppsMarcher/Vecton`, pasta `robo-cargas/` (commit `f17001f` na `main`)

1. O que é e para que serve

Hoje a carga de vendas do Vecton é manual: atualiza-se a planilha `Painel_Vecton.xlsm` (que roda uma query no Protheus), e o arquivo é carregado na tela do Vecton.
O **robô** faz isso sozinho: lê o Protheus (SQL Server `MARCHER-BD`, base `protheus_producao`), calcula o território de cada venda e grava no Vecton (Supabase), **duas vezes ao dia (12:00 e 23:30)**, sempre para o **mês corrente**.

Cada execução apaga o mês corrente no Vecton e regrava tudo (carga completa). Meses anteriores e futuros continuam sendo carregados à mão, como hoje.

2. Como funciona

```
Agendador do Windows (a cada 15 min)  →  robô (Node.js)
   1. entra no Vecton (usuário robo.vecton@marcher.com.br)
   2. lê a configuração (ligada? horários? janela de meses) no próprio Vecton
   3. se há um horário vencido, consulta o Protheus (somente leitura)
   4. resolve o território (tabela Parâmetros → Comercial → Municípios)
   5. cria o lote, o Vecton valida, e o robô aplica
   6. grava o resultado em carga_automatica_log
   7. se der certo: aviso "Carga de Vendas aplicada" (como na carga manual); se der erro: nada é
       aplicado e os administradores recebem e-mail
```

O robô **só lê** do Protheus. Horário, janela de meses e liga/desliga são alterados no Vecton (tabela `carga_automatica_config`), sem mexer no servidor.

Proteções: extração vazia ou queda de mais de 30% nas linhas em relação à última carga → não aplica e avisa. Qualquer linha reprovada na validação (ex.: cliente não cadastrado) → não aplica nada e avisa por e-mail, com as linhas.

3. O que já foi feito e validado

Item	Situação
Código do robô e testes unitários (14 testes)	Pronto, no GitHub
Banco do Vecton (migrations 259, 260, 261, 262): configuração, log, mapa de municípios, aviso por e-mail de erro e de carga concluída	Aplicado
Tela Parâmetros → Comercial → Municípios	Publicada
Usuário `robo.vecton@marcher.com.br` no Vecton	Criado, login validado
Login SQL de leitura `usr_robo_vecton` no `MARCHER-BD`	Criado, validado
Simulação (`--dry-run`): extrai do Protheus sem gravar	Validada: 165 linhas em out/2026, 0 sem território
Carga real (grava e aplica no Vecton)	**Validada** na máquina do Ricardo
Aviso por e-mail em caso de erro	Implementado; ainda não exercitado

Hoje o robô roda **na máquina do Ricardo, manualmente**. Não há tarefa agendada.

4. O que falta — pendências da TI

4.1 Preparar o servidor

- Servidor Windows **ligado 24x7**, com acesso de rede a:
  - `MARCHER-BD` (SQL Server, normalmente porta 1433);
  - internet para `*.supabase.co` (HTTPS, porta 443).
- **Node.js 20 ou superior** instalado.
- Uma **conta do Windows para executar a tarefa** (sugestão: conta de serviço, ex. `MARCHER\svc_robo_vecton`), com permissão de "Fazer logon como tarefa em lote" e leitura/escrita na pasta do robô.
- O driver ODBC do SQL Server **só é necessário** se a autenticação do robô com o banco for pela conta do Windows. Com o login SQL `usr_robo_vecton` (configuração atual) ele não é necessário.

4.2 Levar o robô para o servidor

1. Baixar o repositório do GitHub (`AppsMarcher/Vecton`) e copiar só a pasta `robo-cargas` para uma pasta **local** do servidor, ex.: `C:\robo-cargas`. Não rodar de unidade de rede.
2. Dentro da pasta, rodar:

```
npm install
```

3. Copiar `config.example.json` para `config.json` e conferir (o arquivo **não tem senhas**):

```
"protheus": { "server": "MARCHER-BD", "database": "protheus_producao",
              "auth": "sql", "user": "usr_robo_vecton", "passwordEnv": "PROTHEUS_SENHA" }
```

4. **Testar sem gravar**, na pasta do robô, com as senhas definidas na janela (peça ao Ricardo a senha do Vecton; a do banco é da TI):

```
$env:VECTON_ROBO_SENHA = 'senha-do-vecton'
$env:PROTHEUS_SENHA = 'senha-do-banco'
node src/index.js vendas --dry-run
```

   Resultado esperado: `[SIMULACAO] vendas 2026-10: ... linhas (FAT ..., CART ...); sem territorio: 0; sem tipo: 0` e a frase "nada foi gravado no Vecton".

4.3 Agendar a tarefa

1. Logar no servidor **como a conta de serviço** (as senhas ficam como variáveis de ambiente *do usuário* que roda o script).
2. Abrir o PowerShell **como administrador**, na pasta do robô, e rodar:

```
.\instalar-tarefa.ps1 -Conta "MARCHER\svc_robo_vecton"
```

   O script pede: a senha da conta do Windows, a senha do Vecton e a do SQL Server. Cria a tarefa **"Vecton - Robo de Cargas"**, que dispara a cada 15 minutos; o robô decide sozinho se há horário vencido (12:00 e 23:30).
3. Conferir no Agendador de Tarefas que a tarefa existe e o "Resultado da última execução" é `0x0`.
4. **Desativar a tarefa de teste da máquina do Ricardo** (se existir), para não haver dois robôs carregando o mesmo mês.

4.4 Pasta de cópia dos arquivos (opcional)

O robô grava uma cópia `Vendas_AAAA-MM.csv` em `O:\Controladoria\Gerencial\Vecton\02 - CARGA PAINEL DE VENDAS\3 - CARGAS ENVIADAS`. A conta da tarefa **não enxerga unidades mapeadas** (`O:`): é preciso o **caminho de rede completo (UNC)** e permissão de escrita, ou desligar essa cópia. Se a cópia falhar, a carga **não falha**: o aviso fica registrado no log.

5. Segurança

- O robô **só lê** do Protheus: o usuário `usr_robo_vecton` precisa apenas de `SELECT` nas tabelas `SD2010, SD1010, SB1010, SF2010, SA1010, SC5010, SC6010, SF4010, SYD010`.
- Senhas **nunca** em arquivo nem no repositório: ficam em variáveis de ambiente do usuário da tarefa. O `config.json` está fora do Git.
- O usuário do robô no Vecton tem perfil Administrador (será usado para outras cargas).

6. Como acompanhar e operar

- **Log de execuções** (SQL Editor do Supabase):

```
select started_at, status, reference_month, total_rows, mensagem
from public.carga_automatica_log where job = 'vendas' order by started_at desc limit 10;
```

  Status: `ok`, `erro`, `bloqueada`.
- **Pausar:** `update public.carga_automatica_config set enabled = false where job = 'vendas';` (religar com `true`).
- **Rodar agora, à mão:** `node src/index.js vendas` (apaga e regrava o mês corrente).
- **Quando der erro:** o e-mail chega aos administradores (lista em Parâmetros → Notificações → "Carga automática com erro"). Causa comum: cliente ou produto não cadastrado; basta cadastrar e a próxima execução carrega sozinha.
- Documentação técnica completa: `robo-cargas/README.md`.

7. Pontos em aberto

1. Nome do servidor e da conta de serviço que a TI definir.
2. Caminho de rede (UNC) da pasta `3 - CARGAS ENVIADAS`, ou decisão de desligar a cópia.